Universidad de Murcia
Grado en Ingeniería Informática
Facultad de Informática

Compilador miniC → MIPS
Memoria de prácticas

Compiladores 2025/2026

2º Grado en Ingeniería Informática

3º Programa Académico de Simultaneidad de Doble Titulación con Itinerario específico de Grado en Matemáticas y Grado en Ingeniería Informática

Alumno: Guillermo Fulgencio Parra López
Correo: gf.parralopez@um.es

Alumno: Roberto Martínez Reina
Correo: roberto.m.r@um.es

Convocatoria: Convocatoria II, curso 2025/2026

9 de mayo de 2026

Índice

1. Introducción **3**
1.1. Manual de uso . . . 3

2. Estructura del proyecto **3**
2.1. Organización de carpetas . . . 3
2.2. Ficheros fuente . . . 4
2.3. Estructuras de datos . . . 4

3. Práctica 1: Análisis léxico **5**
3.1. Tokens . . . 5
3.2. Comentarios y cadenas . . . 5
3.3. Recuperación de errores . . . 5

4. Práctica 2: Análisis sintáctico **5**
4.1. Precedencia y asociatividad . . . 5
4.2. Conflictos . . . 6
4.3. Uso de `%code requires` . . . 6
4.4. Recuperación de errores . . . 6

5. Práctica 3: Análisis semántico **7**
5.1. Tabla de símbolos . . . 7
5.2. Control semántico . . . 7
5.3. Generación de la sección `.data` . . . 7

6. Práctica 3: Generación de código MIPS **7**
6.1. Estrategia . . . 7
6.2. Expresiones . . . 8
6.3. Sentencias de control . . . 8
6.4. La función `imprimirCodigo` . . . 8

7. Mejoras del enunciado **9**
7.1. `do-while` (0,5 puntos) . . . 9
7.2. Operadores relacionales (0,5 puntos) . . . 9
7.3. `break` (1 punto) . . . 9

8. Optimizaciones **10**
8.1. Constant folding y materialización perezosa . . . 10
8.1.1. Idea . . . 10
8.1.2. Cambios necesarios en el compilador . . . 10
8.1.3. División por cero en compilación . . . 10
8.1.4. Ejemplo . . . 11
8.2. Eliminación de código muerto . . . 11
8.3. Optimizaciones peephole . . . 11
8.3.1. Qué hace el módulo . . . 11
8.3.2. Cómo se detectan los patrones . . . 12
8.3.3. Por qué marcamos como `#nop` en vez de borrar . . . 12
8.3.4. Integración . . . 12

9. Detalles de implementación **12**
9.1. Declaraciones duplicadas con inicializador 12
9.2. Reseteo del pool de registros en la recuperación de errores 13
9.3. Deduplicación de cadenas literales 13
9.4. Gestión de memoria 13

10. Pruebas y validación **13**
10.1. Estrategia 13
10.2. Cobertura 14
10.3. Cómo ejecutarlas 14
10.4. Algunos resultados concretos 15

11. Ejemplos de funcionamiento **16**
11.1. Ejemplo 1: programa del enunciado 16
11.2. Ejemplo 2: mejoras combinadas 16
11.3. Ejemplo 3: errores semánticos 16

12. Limitaciones conocidas y extensiones futuras **17**

13. Conclusiones **17**

1. Introducción

Esta memoria describe la implementación del compilador miniC → MIPS propuesto en el enunciado de la asignatura. El compilador cubre las cuatro fases habituales (análisis léxico con Flex, análisis sintáctico con Bison, análisis semántico sobre la tabla de símbolos, y generación de código ensamblador MIPS) y añade las tres mejoras puntuadas del enunciado más algunas optimizaciones sobre el código generado, descritas en su sección correspondiente.

Todo el código está escrito en C y se compila con `gcc` sobre Linux. Para validar el código ensamblador producido hemos usado MARS 4.5, que es el simulador recomendado en los recursos del Aula Virtual; el código también funciona en `spim` sin cambios, pero las pruebas y comprobaciones de esta memoria están hechas sobre MARS.

1.1. Manual de uso

Para compilar el compilador y trabajar con él se ofrecen tres objetivos en el `Makefile`:

```
$ make            # genera el ejecutable 'minic'
$ make test       # ejecuta la bateria de pruebas (ver seccion
   10)
$ make clean      # elimina los ficheros generados
```

Una vez construido el ejecutable, se traduce un fichero fuente con extensión `.mc` y se ejecuta el ensamblador resultante en MARS:

```
$ ./minic programa.mc > programa.s
$ java -jar tests/Mars4_5.jar nc programa.s   # modo consola
$ java -jar tests/Mars4_5.jar programa.s      # modo grafico
```

El flag `nc` ("no copyright") desactiva la cabecera de Mars en modo consola, lo cual es útil cuando se captura la salida en un fichero. Si se prefiere `spim`, el comando equivalente es `spim -file programa.s`.

Los errores léxicos, sintácticos y semánticos se envían por `stderr`; el código MIPS se escribe por `stdout`. El compilador solo imprime las secciones `.data` y `.text` cuando los tres contadores de errores están a cero. En cualquier caso, al terminar se vuelca por `stderr` un resumen con el conteo de errores de cada fase.

2. Estructura del proyecto

2.1. Organización de carpetas

El proyecto está organizado en cinco carpetas además de la raíz:

- Raíz: `main.c`, `miniC.l`, `miniC.y`, `Makefile` y `README.md`. El binario `minic` también se deposita aquí al compilar.
- `codigo_aulavirtual/LS/` y `codigo_aulavirtual/LC/`: módulos suministrados por el profesorado (tabla de símbolos y lista de cuádruplas), usados sin modificar.
- `peephole/`: módulo propio con las optimizaciones *peephole*.
- `ficheros_intermedios/`: generado por `make`; aquí van los `.o`, los ficheros de Flex (`lex.yy.c`) y de Bison (`miniC.tab.c`, `miniC.tab.h`, `miniC.output`). Se borra entero con `make clean`.

- `tests/`: contiene `Mars4_5.jar`, el script de pruebas `script_pruebas.sh`, los once ficheros `.mc` en `tests/programas/` y los volcados de las pruebas en `tests/salida/` (subdivididos en `ensamblador/`, `error/` y `out/`).
- `docs/`: esta memoria en PDF.

2.2. Ficheros fuente

Fichero	Tipo	Descripción
`miniC.l`	Flex	Analizador léxico.
`miniC.y`	Bison	Gramática, acciones semánticas y generación de código.
`main.c`	C	Punto de entrada del compilador (abre el fichero y arranca `yyparse`).
`listaSimbolos.h/c`	C	Tabla de símbolos (suministrada por el profesorado).
`listaCodigo.h/c`	C	Lista de cuádruplas MIPS (suministrada por el profesorado).
`peephole.h/c`	C	Módulo de optimizaciones peephole.
`Makefile`	Make	Dependencias y órdenes de compilación.

Cuadro 1: Ficheros del proyecto.

2.3. Estructuras de datos

Las dos estructuras del profesor (`Lista` para símbolos y `ListaC` para código) se usan sin modificar su implementación interna. Hemos añadido dos estructuras auxiliares:

`ExprInfo`. Tipo asociado al atributo del no terminal `expression` para soportar *constant folding*. Contiene la lista de código generada hasta el momento, un valor entero y un flag `es_const` que indica si ese valor está disponible en compilación. Su uso se detalla en la sección 8.1.

Pool de registros temporales. Un `int registros[10]` marca qué registros `$t0`–`$t9` están en uso. `nuevoRegistro()` recorre el array y devuelve el primero libre, y `liberarRegistro()` pone a 0 la posición correspondiente. La tabla de símbolos y los registros temporales son cosas distintas (los temporales no se declaran en miniC, son responsabilidad del compilador), así que un array independiente es lo más simple.

Pila de etiquetas para los bucles. Dos arrays paralelos, `pilaEtiqInicio[MAX_BUCLES]` y `pilaEtiquetasFin[MAX_BUCLES]`, con `MAX_BUCLES = 100`. Se apila al entrar en un `while` o `do-while` y se desapila al salir. La regla de `break` consulta la cima para saber a qué etiqueta saltar. Si se desbordara se aborta con `exit(3)`, lo que prácticamente no ocurre.

3. Práctica 1: Análisis léxico

3.1. Tokens

El analizador reconoce los tokens descritos en el enunciado más los necesarios para las mejoras (`do`, `break` y los operadores relacionales). El orden de las reglas es el que exige Flex para que funcionen el *longest match* y el desempate por posición: las palabras reservadas van antes que la regla de identificador (porque comparten longitud y ganaría la primera escrita), y los operadores de dos caracteres (`==`, `!=`, `<=`, `>=`) van antes que los de uno (`=`, `<`, `>`).

Los identificadores están limitados a 32 caracteres. Si el lexema excede ese máximo se emite error léxico y se trunca con `yytext[32] = '\0'` para que el resto del análisis siga siendo razonable. Las constantes enteras se validan con `strtol`: se comprueba `errno == ERANGE` y el valor frente a 2^{31}.

3.2. Comentarios y cadenas

Comentarios multilínea y cadenas requieren mantener estado, así que usamos condiciones de arranque exclusivas de Flex (`%x COMENTARIO`, `%x CADENA`). En ambos casos guardamos el número de línea de apertura (`inicio_estructura = yylineno`) para que, si llegamos al final del fichero sin cerrar, el mensaje de error pueda apuntar a la línea del `/*` o la `"` que quedó huérfana.

El cuerpo de la cadena se acumula con `yymore()`, que es la forma limpia en Flex de construir el lexema completo sin concatenar manualmente. Las secuencias de escape (`\n`, `\t`, `\r`, `\"`, `\\`) tienen cada una su regla explícita para que la cadena se reconozca como un solo token aunque contenga comillas escapadas.

3.3. Recuperación de errores

Seguimos el esquema clásico de modo pánico. La expresión regular

```
PANICO  [^ \t\n\ra-zA-Z0-9_"=;,+\-*/(){}<>]+
```

captura cualquier secuencia de caracteres que no puedan iniciar un token válido, de modo que ante una entrada como `@#~` el compilador emite un único mensaje en lugar de uno por carácter. Cada error incrementa el contador global `errores_lexicos`, definido en `miniC.y` y declarado como `extern` en el léxico.

4. Práctica 2: Análisis sintáctico

4.1. Precedencia y asociatividad

La gramática del enunciado es ambigua para expresiones, así que hay que declarar precedencias y asociatividades. De menor a mayor:

1. `%nonassoc EQ NEQ LE GE LT GT` — relacionales, no asociativos.
2. `%left MAS MENOS`.
3. `%left POR DIV`.
4. `%precedence SIGNO` — menos unario.
5. `%precedence NOELSE` y `%precedence ELSE` — *dangling else*.

Que los relacionales sean `%nonassoc` significa que `a <b <c` es un error sintáctico; coincide con lo que pide el enunciado ("estos operadores no deben ser asociativos").

El menos unario se marca con la regla `MENOS expression %prec SIGNO`. Usamos `%precedence` en vez de `%left`/`%right` porque es la directiva apropiada para un operador monario: solo importa la prioridad, no hay asociatividad.

Para el *dangling else* usamos `%precedence NOELSE` y `%precedence ELSE` declaradas en ese orden, de modo que el token `else` tiene mayor precedencia que la regla del `if` sin `else` (que se marca con `%prec NOELSE`). El conflicto del *dangling else* se resuelve a favor del *shift* y el `else` se asocia al `if` más cercano, que es lo deseado.

4.2. Conflictos

Ejecutando `bison -d -v miniC.y` generamos el fichero `miniC.output`. La gramática actual no produce ningún conflicto pendiente: todos los conflictos potenciales (precedencias entre operadores aritméticos y relacionales, menos unario frente a binarios, y el *dangling else*) los resuelve Bison en silencio gracias a las directivas de precedencia anteriores. Esto se puede comprobar porque `bison` no emite la línea de aviso "`shift/reduce conflict`"; el resumen final del `.output` no la contiene.

En el `.output` se ven, eso sí, las decisiones tomadas ("`Conflict between rule X and token '+' resolved as reduce`"), pero todas están explícitamente resueltas y no quedan como conflictos pendientes en el resumen. La gramática completa tiene 110 estados.

4.3. Uso de `%code requires`

Un detalle particular de la cabecera del fichero: la definición del tipo `ExprInfo` está dentro de un bloque `%code requires`, no del prólogo `%{ ... %}`. La razón es que `ExprInfo` aparece en la `%union`, y la `%union` se copia al fichero `miniC.tab.h` que incluye el lexer. Si el tipo estuviera en el prólogo, el `miniC.tab.h` no sabría qué es `ExprInfo` y el lexer no compilaría. `%code requires` se copia al `.tab.h` *antes* de la `%union`, justo lo que necesitamos.

Hay un segundo bloque `%code` (sin cualificador) con los prototipos de las funciones que reciben o devuelven `ExprInfo`. Ese bloque va al `.tab.c` después del `#include "miniC.tab.h"`, momento en el que el tipo ya está definido y los prototipos compilan.

4.4. Recuperación de errores

Hay dos reglas de recuperación en `statement`: una sincroniza con el punto y coma (`error PYC`) y otra con la llave de cierre (`error RBRACE`). Las dos hacen `yyerrok` y devuelven una lista de código vacía para que la concatenación ascendente no se rompa.

En una primera versión nos olvidamos de un detalle: si el error ocurre en mitad de una expresión que ya había pedido registros con `nuevoRegistro()`, esos registros quedan marcados como ocupados y nunca se liberan. A la larga se agota el pool y el compilador emite un falso "no hay registros temporales disponibles" en una sentencia que es correcta. Para evitarlo añadimos `resetearRegistros()`, que pone a cero todo el array, y la llamamos en las dos reglas de recuperación. Los registros temporales no tienen que sobrevivir a una sentencia que ha fracasado.

Otro asunto relacionado es la liberación de la memoria asociada a los símbolos descartados durante el modo pánico. Bison ofrece la directiva `%destructor` para esto: cuando

descarta un símbolo de la pila al recuperarse de un error, llama al destructor que le hayamos indicado. Tenemos uno para los `<str>` (`free`) y otro para las `<codigo>` (`liberaLC`), que se encargan de no dejar memoria colgando.

5. Práctica 3: Análisis semántico

5.1. Tabla de símbolos

La tabla se crea en la acción inicial de la regla `program` (siguiendo el estilo de los ejemplos del aula virtual: el `main()` del compilador solo abre el fichero y arranca `yyparse`). Las funciones que usamos sobre ella son:

- `perteneceTS(nombre)` — devuelve si el nombre existe.
- `obtenerTipoTS(nombre)` — devuelve `VARIABLE`, `CONSTANTE` o `CADENA`.
- `insertarTS(nombre, tipo, valor)` — añade un símbolo al final.

Variables y constantes usan el campo `valor` del símbolo para el valor inicial, aunque actualmente las declaraciones siempre se inicializan a 0 en la sección `.data` y la asignación se hace luego con un `li`+`sw`; el campo queda como previsión para la extensión de *folding* a través de identificadores que se comenta en la sección 12. Las cadenas usan `valor` para guardar su número de identificación `$strN`.

5.2. Control semántico

Las comprobaciones del enunciado se reparten entre las reglas de la gramática:

- En `program`: el identificador del programa debe ser `main`.
- En `id_decl`: el identificador no puede estar ya declarado. Además, si el tipo actual es constante, se exige que tenga un inicializador (no se permiten constantes sin valor).
- En la asignación y en `read`: el identificador debe estar declarado y no ser constante. Esta comprobación común se factoriza en `verificarVarAsignable()`.
- En `break`: la variable `nivelBucle` tiene que ser > 0.

Cuando un identificador aparece en una expresión sin haber sido declarado, emitimos el error y además generamos un `li $tN, 0` ficticio en su lugar. Así, aunque el compilador esté en modo de error (no va a producir código real, pero sí continúa el análisis para acumular todos los errores), la maquinaria de registros y atributos no se rompe.

5.3. Generación de la sección `.data`

`imprimirTablaSimbolos()` recorre la tabla dos veces. En la primera pasada escribe las cadenas con su etiqueta `$strN` y la directiva `.asciiz`; en la segunda, las variables y constantes con su etiqueta `_nombre` y la directiva `.word 0`. Separar las pasadas evita preocuparse por el orden en que aparecen unas y otras en el fuente.

6. Práctica 3: Generación de código MIPS

6.1. Estrategia

Hemos seguido la primera opción del enunciado: a cada no terminal principal de la gramática le asignamos como atributo una `ListaC`. En cada reducción se concatenan las

listas de los componentes y se añaden las instrucciones MIPS específicas de esa regla. Al reducir `program` ya tenemos la lista completa, que se pasa por el módulo de *peephole* y se imprime.

El atributo de `expression` es distinto: en vez de una `ListaC` a secas usamos un `ExprInfo` con tres campos, lo que permite saltar la generación de un `li` cuando el valor ya se conoce en compilación (sección 8.1).

6.2. Expresiones

Cada operador binario (`+`, `-`, `*`, `/` y los seis relacionales) produce la misma estructura: se evalúan los dos operandos, se concatenan sus códigos, se pide un registro nuevo y se emite la instrucción MIPS correspondiente con `regRes`, `reg1`, `reg2`. Los registros de los dos operandos se liberan al final. El patrón se repetía diez veces, así que lo extrajimos en una función auxiliar:

```
ExprInfo generarOpBinaria(ExprInfo izq, ExprInfo der,
                          const char *mipsOp,
                          int valorFold, int plegable);
```

Las diez reglas de expresión binaria pasan a ser una llamada a esta función con el mnemónico MIPS (`."add"`, `"seq"`, etc.) y el cálculo constante que se usaría si ambos operandos fueran constantes. El *constant folding* se hace dentro de la función.

El menos unario se traduce con `neg`. Si el operando es constante, plegamos en compilación (`-e.valor`) sin emitir nada.

6.3. Sentencias de control

`if` sin `else`. Código de la condición, `beqz $reg, $etiqFin`, cuerpo, etiqueta de fin.

`if-else`. Condición, `beqz $reg, $etiqElse`, cuerpo del *then*, `b $etiqFin`, etiqueta del *else*, cuerpo del *else*, etiqueta de fin. El salto incondicional tras el *then* es imprescindible para no caer en la rama *else*.

`while`. Etiqueta de inicio, condición, `beqz` al fin, cuerpo, `b` al inicio, etiqueta de fin.

`do-while`. Etiqueta de inicio, cuerpo, condición, `bnez` al inicio, etiqueta de fin. No hay `beqz` antes del cuerpo (el cuerpo siempre se ejecuta una vez) y se usa `bnez` en lugar de `beqz`.

`print` y `read`. Para cada entero en un `print` se emite `move $a0, $reg` + `li $v0, 1` + `syscall`; para cada cadena, `la $a0, $strN` + `li $v0, 4` + `syscall`. Un `read` es `li $v0, 5` + `syscall` + `sw $v0, _var`.

6.4. La función `imprimirCodigo`

Recorre la lista de cuádruplas. Los nodos con `op = "label"` se imprimen como `nombre:` al comienzo de línea. Los nodos con `op = "#nop"` (pseudo-instrucción interna que generamos al eliminar instrucciones redundantes desde el peephole; ver sección 8.3) se ignoran. El resto se imprime como `\top res, arg1, arg2`, omitiendo los campos `NULL`. Las cabeceras

de sección (`# Seccion de datos`, `# Seccion de codigo`, `# Fin`) se emiten directamente desde `finalizar_programa` para reproducir el formato del enunciado.

7. Mejoras del enunciado

7.1. `do-while` (0,5 puntos)

Se añade el token `DO` al léxico y una regla nueva en `statement`:

`statement` → `do statement while ( expression ) ;`

La traducción MIPS es la descrita en la sección anterior. Para la gestión de `break` reutilizamos el mismo mecanismo que en `while`: se apila la etiqueta de fin al entrar y se desapila al salir. Internamente, ambas reglas llaman a `iniciar_bucle()` y `finalizar_bucle()`, que encapsulan ese estado.

7.2. Operadores relacionales (0,5 puntos)

Los seis operadores se traducen a las pseudoinstrucciones MIPS de tipo *set-on-condition*:

miniC	**MIPS**	**Descripción**
`==`	`seq`	Set if equal
`!=`	`sne`	Set if not equal
`<`	`slt`	Set if less than
`>`	`sgt`	Set if greater than
`<=`	`sle`	Set if less or equal
`>=`	`sge`	Set if greater or equal

Cuadro 2: Traducción de los operadores relacionales.

Cada una de estas pseudoinstrucciones deja un 1 o un 0 en su registro destino, así que se pueden usar sin más como condición de un `beqz`/`bnez`. Las reglas de expresión para estos seis operadores pasan por la misma `generarOpBinaria` que los aritméticos, lo que las dejó en una línea cada una una vez hecho el refactor.

7.3. `break` (1 punto)

La regla es directa: `break PYC`. La parte delicada es la gestión del anidamiento. Usamos dos estructuras:

- `nivelBucle` (entero). Se incrementa al entrar en un bucle y se decrementa al salir. Si vale 0 cuando se encuentra un `break`, se emite el error semántico "break fuera de bucle".
- `pilaEtiquetasFin` (array). Contiene la etiqueta de fin del bucle actualmente activo. El `break` se traduce a `b pilaEtiquetasFin[topeBucles]`, es decir, salta al fin del bucle más interno.

Al salir del bucle se desapila automáticamente. El esquema funciona para cualquier nivel de anidamiento y combina `while` y `do-while` sin diferencia (ver fichero `10_break_anidado.mc` en la sección de pruebas).

8. Optimizaciones

8.1. Constant folding y materialización perezosa

8.1.1. Idea

La idea es evaluar en compilación cualquier expresión cuyos operandos sean constantes conocidas, evitando emitir `li`/`add`/`mul`/... que después MARS tendría que ejecutar. Por ejemplo, `3 + 4 * 2` debería reducirse a una sola instrucción `li $t0, 11` en lugar de cinco.

La forma natural de implementarlo es enriquecer el atributo del no terminal `expression`. En vez de asociar solo una `ListaC`, usamos un `struct`:

```
typedef struct {
    ListaC codigo;      /* codigo emitido hasta ahora */
    int    valor;       /* valor conocido en compilacion */
    int    es_const;    /* 1 si 'valor' es valido, 0 si no */
} ExprInfo;
```

Cuando `es_const == 1` la lista `codigo` está vacía y `valor` contiene el resultado: ningún `li` se genera todavía. Solo si ese valor acaba usándose realmente (por ejemplo como operando de una operación con otro valor no constante, o como argumento de `print`), se "materializa": en ese momento se pide un registro, se emite el `li $reg, valor` y el flag se pone a 0. Esto es lo que en el correo previo con el profesor llamamos *materialización perezosa*: retrasar al máximo la asignación de un registro y evitarla en constantes que no se llegan a usar.

8.1.2. Cambios necesarios en el compilador

El léxico no cambia. En el sintáctico se declara `%type <expr>expression` y los demás no terminales siguen con `<codigo>`. Las acciones semánticas de las reglas binarias siguen el mismo patrón:

1. Si ambos operandos son constantes, plegar: crear una `ExprInfo` con `es_const = 1` y el valor calculado.
2. Si no, materializar los operandos constantes que haya (emitiendo su `li`), concatenar los dos códigos, emitir la instrucción MIPS y devolver una `ExprInfo` con `es_const = 0`.

`materializarExpr()` se encarga de pedir un registro, emitir `li $reg, valor`, anotar el registro como resultado con `guardaResLC` y bajar el flag. Se llama desde todas las reglas que consumen una expresión: asignación, inicialización, argumento de `print`, y condiciones de `if`/`while`/`do-while` (en este último solo si la condición no es constante; si lo es, se gestiona en la sección 8.2).

8.1.3. División por cero en compilación

Una consecuencia del *folding* es que `100 / 0` se detecta en tiempo de compilación. Emitimos un aviso (no un error, porque el lenguaje no prohíbe escribir esa expresión) y sustituimos el resultado por 0 para no romper el resto del proceso. Por ejemplo, para el fichero `05_folding.mc` que contiene `var int div_cero = 100 / 0;` en la línea 9, la salida por `stderr` es:

```
Aviso en linea 9: division por cero en expresion constante.
```

8.1.4. Ejemplo

Entrada:

```
void main() {
    var int x = 3 + 4 * 2;
    print(x, "\n");
}
```

MIPS generado:

```
    li $t0, 11
    sw $t0, _x
    lw $t0, _x
    move $a0, $t0
    li $v0, 1
    syscall
    ...
```

Sin *folding* la inicialización de `x` generaría 5 instrucciones (`li`, `li`, `mul`, `li`, `add`); con *folding* queda una sola.

8.2. Eliminación de código muerto

Con el flag `es_const` las reglas de `if`, `if-else`, `while` y `do-while` pueden inspeccionar la condición y decidir si hace falta generar código. Los casos:

- `if (0) S` $\longrightarrow$ lista vacía.
- `if (K) S` con $K \neq 0$ $\longrightarrow$ solo el cuerpo, sin condición ni etiquetas.
- `if (0) S1 else S2` $\longrightarrow$ solo `S2`.
- `if (K) S1 else S2` con $K \neq 0$ $\longrightarrow$ solo `S1`.
- `while (0) S` $\longrightarrow$ lista vacía.
- `while (K) S` con $K \neq 0$ $\longrightarrow$ bucle infinito sin comprobación; solo es útil si el cuerpo contiene un `break`, pero el compilador no se mete a verificar eso.
- `do S while (0)` $\longrightarrow$ se genera el cuerpo una sola vez, sin salto de retorno.

La combinación con los relacionales lleva a casos donde el usuario escribe algo claramente muerto sin darse cuenta, como `if (3 >5) ...`. La regla de `>` pliega `3 >5` a 0, y el `if` de arriba ve una condición constante igual a 0 y descarta la rama.

8.3. Optimizaciones peephole

8.3.1. Qué hace el módulo

Son transformaciones locales sobre el código MIPS *ya generado*, recorriendo la lista de cuádruplas y buscando patrones reconocibles. Están en un módulo aparte (`peephole.c`, `peephole.h`) con un único punto de entrada, `peephole_optimize(ListaC)`, que se llama una vez antes de imprimir el código en `finalizar_programa`.

Los seis patrones que reconocemos:

1. `add R, X, 0` y `add R, 0, X` $\longrightarrow$ `move R, X`.
2. `sub R, X, 0` $\longrightarrow$ `move R, X`.
3. `mul R, X, 1` y `mul R, 1, X` $\longrightarrow$ `move R, X`.
4. `mul R, X, 0` (y el simétrico) $\longrightarrow$ `li R, 0`.
5. `mul R, X,` 2^k (y el simétrico) $\longrightarrow$ `sll R, X, k`.
6. `move R, R` $\longrightarrow$ `#nop`.

8.3.2. Cómo se detectan los patrones

Cuando el código MIPS contiene `add $t3, $t1, $t2`, el módulo no sabe por sí solo si `$t2` contiene un 0 o no. Lo averigua haciendo un barrido lineal de la lista desde el inicio hasta el punto del `add`. Mantiene el valor del último `li $t2, _` encontrado y lo invalida si en el camino aparece otra instrucción que escriba en `$t2`. Cuando llega al `add`, si el valor sigue siendo válido, aplica la transformación.

Hay dos sutilezas en la búsqueda. La primera: una etiqueta (`op = "label"`) rompe el bloque básico (hay un salto entrante que no se puede ver localmente) y por tanto invalida cualquier `li` previo. La segunda: la propia pseudo-instrucción `#nop` que dejamos atrás al eliminar un *move* redundante no debe contar como "reescritura del registro". Las dos comprobaciones están explícitas en `buscar_ultimo_li`.

8.3.3. Por qué marcamos como `#nop` en vez de borrar

La implementación suministrada de `ListaC` no expone ninguna operación para suprimir nodos. Añadirla no era complicado, pero implicaba modificar `listaCodigo.c`. La alternativa que adoptamos, marcar la operación como `#nop` y filtrarla en `imprimirCodigo`, evita tocar los ficheros del Aula Virtual y produce el mismo resultado.

8.3.4. Integración

En `finalizar_programa`, después de reducir el cuerpo, se invoca `peephole_optimize(cuerpo)` y a continuación `imprimirCodigo(cuerpo)`. Una sola pasada. En la sección 12 comentamos por qué no ejecutamos el peephole hasta un punto fijo.

9. Detalles de implementación

9.1. Declaraciones duplicadas con inicializador

La regla `id_decl : ID ASIG expression` puede fallar de dos maneras: porque el identificador ya estuviera declarado, o por un error en la expresión. El caso duplicado requiere cuidado: si simplemente llamamos a `generarAsignacion($3, $1)` como con las declaraciones bien formadas, emitiríamos un `sw $reg, _nombre` hacia una etiqueta que *no va a existir* en la sección `.data` (porque `_nombre` no se reinsertaría en la tabla). En un programa con errores eso no importa (no se va a imprimir el código), pero es código sucio.

Lo separamos así:

```
if (duplicado) {
    $$ = creaLC();
    liberaLC($3.codigo);   /* descartamos el codigo ya generado
        */
    free($1);
} else {
    $$ = generarAsignacion($3, $1);
    insertarTS($1, tipoActual, 0);
}
```

Cuando hay duplicado, la expresión ya generó su código antes de que el parser redujera esta regla (es un analizador ascendente; `$3` ya está construido). Hay que liberarlo explícitamente para no dejar basura colgada en la lista.

9.2. Reseteo del pool de registros en la recuperación de errores

Ya lo comentamos en la sección 4.4 pero merece mencionarse aquí porque pertenece a la misma familia de problemas. En la batería de pruebas de errores sintácticos se notaba enseguida: antes de añadir `resetearRegistros()`, en programas con tres o cuatro errores en cascada llegaba un momento en el que el compilador devolvía "no hay registros temporales disponibles". Tras añadirlo, eso ya no ocurre.

9.3. Deduplicación de cadenas literales

Para evitar duplicar memoria en la sección `.data`, no emitimos cadenas repetidas. Si el usuario escribe `print("hola");` en dos lugares distintos, la regla de `STRING` busca primero el lexema en la tabla de símbolos. Si ya está, se reutiliza su `$strN` y se libera el `strdup` duplicado del lexer; si no, se inserta como cadena nueva. Aprovecha la tabla de símbolos para algo más que comprobar tipos.

9.4. Gestión de memoria

Durante el desarrollo cuidamos de no acumular fugas masivas. Dos puntos a destacar:

Primero, la versión inicial de `nuevoRegistro()` usaba `asprintf` para construir la cadena `"$tN"` en cada llamada, lo que dejaba memoria sin liberar continuamente. Lo cambiamos a un array estático `static const char *nombres_registros[10]` y la función pasa a devolver punteros inmutables. Solo con eso desaparece la fuga más frecuente del compilador.

Segundo, los `%destructor` de Bison que ya hemos comentado: cuando el parser entra en modo pánico y descarta tokens de la pila, las cadenas `strdup`'eadas en el lexer y los subárboles de `ListaC` se liberan automáticamente. Sin esto, cada error sintáctico filtraría memoria.

Sí persisten algunas fugas menores: las cadenas formateadas dinámicamente para MIPS (etiquetas `$lN`, nombres `_x`) se insertan en `ListaC` y `liberaLC` no las libera, porque la estructura mezcla literales inmutables de C con punteros dinámicos sin forma de distinguirlos. El impacto es de unos pocos KB que el SO recupera al cerrar el proceso. Lo dejamos anotado por transparencia.

10. Pruebas y validación

10.1. Estrategia

Al principio teníamos un único fichero de pruebas enorme que tocaba todos los aspectos del lenguaje. Resultó ser muy mala idea: con un error léxico a propósito en la primera línea, el compilador se negaba (correctamente) a generar el `.s`, y todas las pruebas de generación de código que estaban después no se podían verificar. Además era muy difícil interpretar la salida: errores reales, errores inducidos a propósito, avisos del compilador y salida de MARS, todo mezclado.

La segunda versión, que es la que entregamos, son once ficheros pequeños con un propósito único cada uno. Los que no deben producir `.s` (los que incluyen errores a propósito) están claramente separados de los que sí. La especificación detallada del contrato de cada test — cuántos errores se esperan en cada fase, qué entrada de `stdin` hay

que alimentar a los que leen, y qué subcadena tiene que aparecer en la salida de MARS — está en la tabla de invocaciones del script `tests/script_pruebas.sh`. De esta forma todo el "contrato" del test queda en un único sitio y los `.mc` se mantienen limpios y legibles.

10.2. Cobertura

Fichero	Tipo	Qué prueba	Errores	MARS
`01_enunciado.mc`	Válido	Ejemplo del enunciado (regresión)	0	Sí
`02_lexico_err.mc`	Inválido	Overflow, id >32, `@#~`, `!` aislado	4 léxicos + 2 sint. en cascada	No
`03_sintactico_err.mc`	Inválido	`;` ausente, `)` sobrante, `if` sin parens, `=` sin expr.	4 sintácticos (+1 sem. en cascada)	No
`04_semantico_err.mc`	Inválido	`id`≠`main`, redecl., const reasignada, uso sin declarar, `read(const)`, `break` fuera	6 semánticos	No
`05_folding.mc`	Válido	Constant folding y división por 0	0 (1 aviso)	Sí
`06_dead_code.mc`	Válido	`if(0)`, `while(0)`, `if(K)`, `if(K) else`	0	Sí
`07_peephole.mc`	Válido	Los 6 patrones del peephole	0	Sí (lee 1 entero)
`08_relacionales.mc`	Válido	Los 6 operadores en condición cierta y falsa	0	Sí
`09_do_while.mc`	Válido	`do-while` estándar, `do-while(0)`, con `break`	0	Sí
`10_break_anidado.mc`	Válido	`break` en 2 y 3 niveles de anidamiento	0	Sí
`11_valido_completo.mc`	Válido	Integración: suma, factorial, potencia de 2	0	Sí (lee 1 entero)

Cuadro 3: Batería de pruebas. La columna "Errores" indica los esperados; la columna "MARS" indica si genera `.s` ejecutable.

10.3. Cómo ejecutarlas

La forma recomendada es a través del `Makefile`:

```
$ make test
```

Esto compila el compilador si hace falta, ejecuta `tests/script_pruebas.sh` sobre `tests/programas/`, compara los contadores de errores con los esperados y, si hay `java` en el sistema y `tests/Mars4_5.jar` accesible, ejecuta cada `.s` válido en MARS verificando una subcadena de la salida. El script imprime PASS o FAIL para cada fichero y devuelve

un código de salida no nulo si alguno falla. Las salidas se reparten en tres subdirectorios de `tests/salida/`:

- `ensamblador/<nombre>.s` — código MIPS producido por el compilador.
- `error/<nombre>.err` — todo lo que el compilador escribió por `stderr` (avisos, errores y resumen final).
- `out/<nombre>.out` — salida de MARS al ejecutar el `.s` correspondiente.

Para ejecutar un test concreto a mano, los ficheros que generan `.s` se invocan así:

```
$ ./minic tests/programas/01_enunciado.mc > 01.s
$ java -jar tests/Mars4_5.jar nc 01.s
```

Los que leen entrada se redirigen:

```
$ ./minic tests/programas/07_peephole.mc > 07.s
$ echo "2" | java -jar tests/Mars4_5.jar nc 07.s
```

Los ficheros con errores se ejecutan simplemente con:

```
$ ./minic tests/programas/04_semantico_err.mc
```

y se comprueba que los mensajes en `stderr` y los contadores finales coinciden con lo que indica la fila correspondiente de la tabla anterior.

10.4. Algunos resultados concretos

Para `01_enunciado.mc`, MARS reproduce literalmente la salida del enunciado:

```
Inicio del programa
c = 5
c = 4
c = 3
c = 2
c = 1
Final
```

Para `11_valido_completo.mc` con entrada $N = 5$:

```
Introduce N: Suma 1..5 = 15
5! = 120
Primera potencia de 2 >= 5: 8
Fin
```

Para `04_semantico_err.mc`, la salida relevante está en `stderr`:

```
Error semantico en linea 3: id de programa incorrecto
Error semantico en linea 5: x ya declarada
Error semantico en linea 7: PI es constante
Error semantico en linea 8: y no declarada
Error semantico en linea 9: PI es constante
Error semantico en linea 10: break fuera de bucle

Errores lexicos: 0
Errores sintacticos: 0
Errores semanticos: 6
```

No se genera código, como se espera.

11. Ejemplos de funcionamiento

11.1. Ejemplo 1: programa del enunciado

La entrada y la salida son las del enunciado (sección anterior). Lo interesante del `.s` generado es que la expresión `5+2-2` se reduce a un único `li $t0, 5` gracias al *constant folding*: sin la optimización esa inicialización produciría cinco instrucciones (dos `li`, un `add`, otro `li`, un `sub`); con la optimización, una.

11.2. Ejemplo 2: mejoras combinadas

```
void main() {
    var int x;
    var int y = 2 * 4 + 1;
    read(x);
    do {
        if (x < 0) break;
        print("x = ", x, "\n");
        x = x - 1;
    } while (x > 0);
    if (3 > 5) print("imposible\n");
    else print("correcto\n");
    while (0) { y = y + 100; }
    print("y = ", y, "\n");
}
```

Listing 1: Entrada: ejercita do-while, break, relacionales y las optimizaciones.

Qué pasa con cada trozo:

- `2 * 4 + 1` se pliega a `li $t0, 9`. No aparece ningún `mul` ni `add`.
- El `do-while` con `break` funciona: cuando `x <0` (`slt` seguido de `beqz`), el `break` emite un `b $l`*`fin`* al fin del `do-while`.
- `x <0` genera `slt`, `x >0` genera `sgt`.
- `3 >5` se pliega a 0. El `if-else` con condición constante falsa descarta el *then* y deja solo el *else*. En el `.s` no aparece ningún `la $a0, $str`*`N`* cargando la cadena "`imposible`".
- `while (0)` desaparece entero.

11.3. Ejemplo 3: errores semánticos

```
void principal() {
    var int x, x;
    const int c = 10;
    c = 5;
    y = 3;
    break;
}
```

Listing 2: Entrada con errores semánticos.

Salida por `stderr`:

```
Error semantico en linea 1: id de programa incorrecto
Error semantico en linea 2: x ya declarada
Error semantico en linea 4: c es constante
Error semantico en linea 5: y no declarada
Error semantico en linea 6: break fuera de bucle

Errores lexicos: 0
Errores sintacticos: 0
Errores semanticos: 5
```

Los cinco errores se reportan, no solo el primero. El compilador no aborta al primer fallo: continúa con el análisis para dar un informe completo. No se emite código MIPS.

12. Limitaciones conocidas y extensiones futuras

Cosas que sabemos que se podrían mejorar y que dejamos fuera del alcance de esta entrega:

Folding a través de constantes declaradas. El profesor Eduardo sugirió en su correo de respuesta que `Simbolo` podría ampliarse con un flag que indique si el valor del identificador está disponible en compilación, de manera que `const int c = 5; var int x = c * 2;` pliegue `c * 2` a 10. Nuestra implementación trata siempre los identificadores como no constantes (`es_const = 0` en la regla de `ID`). Es conservador y correcto, pero deja sobre la mesa un caso frecuente. Implementarlo requeriría: añadir un campo booleano a `Simbolo`; al insertar una constante con valor literal, marcarlo a 1; invalidarlo si el símbolo aparece alguna vez como destino de `read`; y en la regla de `ID`, devolver una `ExprInfo` constante cuando el flag esté activo.

Peephole en punto fijo. La pasada del peephole se ejecuta una sola vez. En algunos casos una transformación expone una nueva: por ejemplo, si una `sub` se convierte en `move` y eso deja un `move $r, $r`, el *move redundante* no se elimina (no hay segunda pasada). Envolver la llamada en un bucle `do ...while (cambios)` hasta convergencia cubriría estos casos.

***Spilling* de registros.** El pool fijo de diez `$tN` es suficiente para miniC, pero formalmente es una limitación: una expresión con once subexpresiones vivas simultáneas agotaría el pool. En miniC real no ocurre.

Liberación total de cadenas dinámicas. Comentado en la sección de gestión de memoria. Las etiquetas y nombres de variable formateados con `asprintf` no se liberan porque la `ListaC` no distingue literales de heap. La vía natural sería un *string arena allocator* con la misma vida que el compilador.

13. Conclusiones

Hemos implementado un compilador de miniC a MIPS funcional, que cubre la parte obligatoria del enunciado y las tres mejoras puntuadas (`do-while`, operadores relacionales,

`break`). Sobre esa base hemos añadido cuatro optimizaciones: *constant folding* con materialización perezosa apoyado en un atributo sintetizado con flag, eliminación de ramas muertas con condiciones constantes, deduplicación de cadenas literales, y un módulo *peephole* separado que reconoce seis patrones.

El código pasa las once pruebas descritas en la sección 10. La ejecución en MARS del código generado coincide con el comportamiento del programa fuente en todos los casos válidos, y los errores deliberados en los ficheros de prueba correspondientes se detectan y reportan con línea y descripción.

Hemos disfrutado del proceso a la par que aprendido sobre Compiladores.